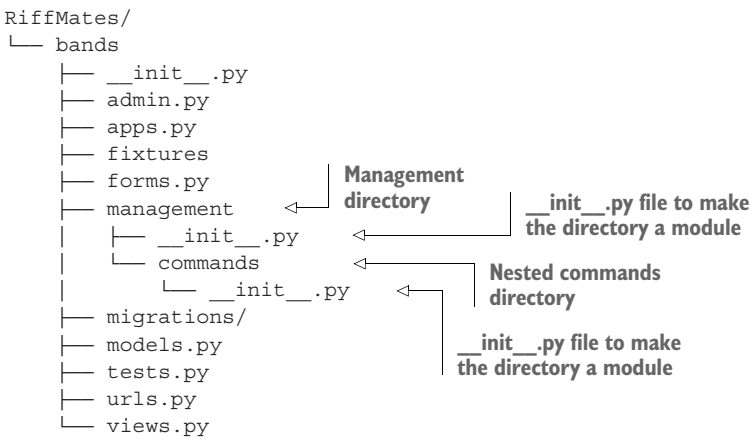
To write a command, you simply need to put the right thing in the right place in your app. The "right thing" is a class in a file, with the file name dictating the name of the management command. The "right place" is a bit weird. Django looks for management command files in your app directory inside nested subdirectories, called *management/commands*. I've always figured that this was part of some deeper long-term plan, that there were going to be other kinds of "management" tools, and thus a nested directory would be needed. Whatever the reason for this otherwise empty structure, that's what Django expects.

Let's start by writing a management command that outputs all `Musician` objects in the database to the terminal. As the `Musician Model` lives in the `bands` app, it makes sense to put the command in there as well. The first thing you need to do is create the nested management/commands directory structure. And as commands are Python files, you also need to create empty __init__.py files to mark the directories as modules. The terminal session in the following listing shows you how to do just that.

Listing 9.1 The shell commands used to create command directories

```
(venv) RiffMates$ mkdir bands/management
(venv) RiffMates$ mkdir bands/management/commands
(venv) RiffMates$ touch bands/__init__.py
(venv) RiffMates$ touch bands/commands/__init__.py
```

Having performed these actions, you'll now have the following contents in the `bands` app directory:



With the structure in place, it is time to create the file. To make a management command called `musicians`, you need a file in the commands directory called musicians.py. Inside that file, you create a class that inherits from `django.core.management.base.BaseCommand` (another unnecessary level of nesting—the Django developers must have been on an organizational bender). Django doesn't care what you call your class, as long as you inherit from `BaseCommand`. As the file name is what